

Mini D9 — Software Specification

Sep 29, 2026 · @Jonathan

Overview

The D9 is an autonomous tracked model (about 25×15 cm) that clears paths, detects the enemy marker, and tows the APC back to safety. It is one of five parts of a larger multi-vehicle system; our team owns the D9, and this spec covers its software only.

In scope: navigation to server-given coordinates, obstacle pushing, towing with an electromagnet, red LED detection, status reporting, and sand scooping for the berm (mechanism still in design). Out of scope: the server, the drone, the tank, the APC, and HQ, which other teams build.

Development approach (option C): logic is prototyped and tested in ROS 2 on a laptop, then ported to the robot's microcontroller. Core logic is written as plain C++ so the port reuses it unchanged.

System context

All communication passes through the central server; vehicles never talk to each other directly. The drone locates every vehicle and publishes positions in a coordinate system relative to the model area.

Party	Built by	What it exchanges with the D9
Server	Server team	Sends task commands (clear, tow) and forwarded positions; receives all D9 reports
Drone (AVATA 1)	Drone team	Supplies the D9's position (via server); receives step 4 "path cleared, tow done"
APC	APC team	Step 2 "unloaded, stopped in depth" triggers the tow; physically towed by the D9
Tank	Tank team	Receives step 3: arrival location data and its position coordinates
HQ (rear)	Server/HQ team	Receives D9 position every 30 s and the enemy detection time

The tank must leave within 3 seconds of enemy identification, and the clock starts from the D9's detection message. This makes detection reporting the most time-critical path in the D9 software.

Functional requirements

The D9 has six capabilities, each testable on its own; the mission order is wired together last.

ID	Requirement	Trigger	Done when
FR-1	Navigate to a target coordinate	Any task needing movement	Within position tolerance of the target
FR-2	Clear obstacles: drive through the target about 15 cm, reverse out, optionally repeat at another angle	<code>clear</code> command with x, y	Push pattern complete, <code>done</code> sent
FR-3	Tow the APC: approach, final alignment, magnet on, drive to safe spot, magnet off	<code>tow</code> command with APC x, y and safe-spot x, y	APC released at safe spot, <code>done</code> sent

ID	Requirement	Trigger	Done when
FR-4	Detect the red LED and report it immediately	Camera sees red blob above threshold	<code>enemy</code> message sent with D9 position and timestamp
FR-5	Report own position	Every 30 s	<code>position</code> message sent
FR-6	Emergency stop: motors off, magnet state held, task aborted	<code>stop</code> command, at any time	Motors stopped within one loop cycle

Berm building (scoop sand, carry, dump) is a later requirement, FR-7, specified once the bucket mechanism is designed.

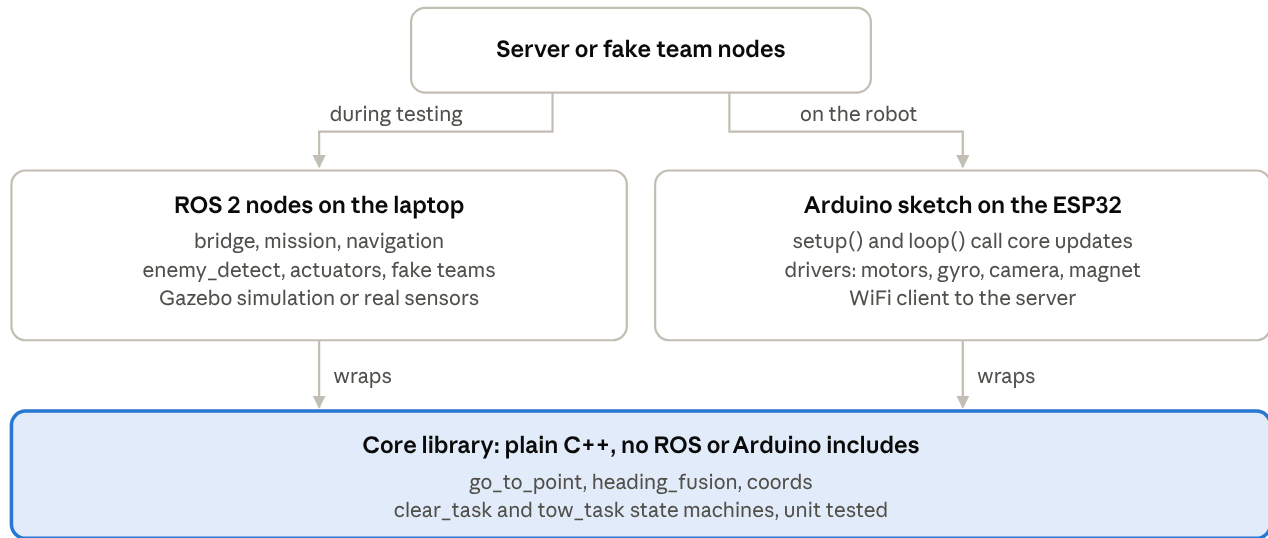
Non-functional requirements

- **Latency:** the `enemy` message leaves the D9 as soon as detection is confirmed, never batched with periodic reports. Target: under 500 ms from detection to send, leaving margin inside the 3-second rule.
- **Non-blocking:** no `delay()` or blocking waits anywhere. Every module runs an `update()` step from the main loop, timed with `millis()`.
- **Safety:** a stop command always wins. Motors stop if no position or command arrives for a timeout period (value to be set during testing).
- **Portability:** core logic in plain C++ with no ROS or Arduino includes, so the same files compile in ROS nodes and the Arduino sketch.
- **Isolation of unknowns:** protocol, coordinate system, and hardware each sit behind one interface, so a decision by another team changes one file.
- **Testability:** every capability can run against fake inputs on a laptop before hardware exists.

Software architecture

All decision-making lives in one core library; ROS nodes and the Arduino sketch are thin wrappers that feed it inputs and carry out its outputs.

One core library runs in both the ROS test setup and the robot



software layers · core library shared by ROS and the robot

The ROS nodes each own one job:

- **bridge** : translates between ROS topics and the server protocol (a fake publisher until the protocol is decided).
- **mission** : receives commands, runs the clear and tow state machines, sends acks and **done** .
- **navigation** : fuses drone positions with the gyro and drives toward the current target.
- **enemy_detect** : watches the camera for the red LED and publishes **enemy** immediately.
- **actuators** : turns motor, magnet and bucket commands into simulator or hardware output.

Localization and heading

Position (x, y) comes from the drone via the server; heading comes from an onboard gyro, corrected by drone positions. The drone is not expected to supply heading.

1. **Initial heading**: at start, drive straight about 20 cm and compute heading from the two drone positions: $\text{heading} = \text{atan2}(y_2 - y_1, x_2 - x_1)$.
2. **Tracking**: the gyro (MPU6050 or BNO085, gyro only) integrates turns from that reference.

3. **Drift correction:** whenever the D9 drives straight past a minimum distance, recompute heading from drone positions and blend it into the gyro estimate.
4. **Between updates:** position is estimated from the last drone fix plus commanded motion, until the next fix arrives.

No magnetometer: the electromagnet and motors would corrupt compass readings. If the drone team can detect two roof markers (front and back), drone-supplied heading replaces step 1 and the gyro becomes a backup.

For the final few centimetres of the tow approach, a rear distance sensor (ultrasonic or IR) guides alignment, since drone accuracy is likely too coarse for the magnet to make contact.

Communication interfaces

Inside our code, everything talks over ROS 2 topics; a single bridge node translates to the server's real protocol once it is decided. Until then, fake nodes publish and consume these topics. MQTT is our proposal to the server team.

Topic	Direction	Fields	Rate
/d9/pose	In (drone via server)	x, y (cm), stamp	Drone update rate (unknown)
/d9/cmd/clear	In	x, y	On demand
/d9/cmd/tow	In	apc_x, apc_y, safe_x, safe_y	On demand
/d9/cmd/stop	In	none	On demand
/d9/status/position	Out	x, y, heading	Every 30 s
/d9/status/enemy	Out	x, y, stamp	Immediately on detection
/d9/status/done	Out	task name	On task completion
/d9/status/ack	Out	command name	On every command received

Draft wire format for the server, in JSON:

```

{"type": "clear", "x": 200, "y": 80}
{"type": "tow", "x": 150, "y": 60, "to_x": 20, "to_y": 20}
{"type": "enemy", "x": 130, "y": 50, "t": 1727612345}

```

Steps 3 (to tank) and 4 (to drone) from the system diagram will be added as outgoing topics once their fields are agreed.

Task state machines

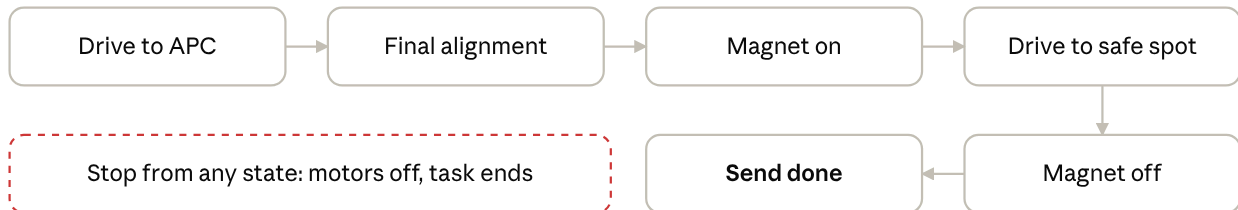
Each command starts one task state machine; only one task runs at a time, and a new command while busy is rejected with a `busy` ack.

Both tasks end by reporting done; a stop aborts either one

Clear task (FR-2)



Tow task (FR-3)



clear and tow task state machines

Enemy detection is not a task: it runs every loop cycle in parallel with whichever task is active, and never waits for it. Each state has a timeout; on timeout the task ends, motors stop, and a failure `done` is reported so the server can react.

Testing strategy

Testing runs in four stages, each usable before the next exists.

1. **Unit tests (laptop, no ROS):** core library tested with GoogleTest: heading math, go-to-point decisions, state machine transitions, tolerance checks.
2. **Simulation (ROS 2 + Gazebo):** a box model of the D9 with differential drive and a camera, a glowing red object as the enemy, pushable obstacles, and an APC model attached through a detachable joint to fake the magnet.

3. **Fake-team nodes:** `fake_server` sends commands, `fake_drone` publishes noisy x, y without heading, `fake_hq` checks the 30 s reports and the enemy timing.
4. **Hardware:** the real D9 on the floor, commands from the fake nodes on the laptop, then fakes replaced by real teams one at a time.

Every bug found on hardware gets a matching unit or simulation test before it is fixed.

Assumptions and open questions

Each assumption below lives in one place in the code and can change without touching the rest.

Assumption	Lives in	Owner to confirm
Internal units are cm and degrees, 0° along +x, counter-clockwise positive	<code>coords</code> module	Drone team
Drone gives x, y only, no heading	<code>localization</code>	Drone team
Enemy report carries the D9's position at detection, not the LED's	<code>enemy_detect</code>	Server/HQ team
Server protocol is MQTT with JSON payloads	<code>bridge</code> node	Server team
Magnet is a single on/off output	<code>actuators</code>	Mechanical team
Final controller is an ESP32	Arduino port	Our team

Open questions:

- ☐ How often does the drone publish positions, and how accurate are they?
- ☐ Where is the origin of the coordinate system, and which way is 0°?
- ☐ What fields do steps 3 (to tank) and 4 (to drone) carry?
- ☐ Where on the APC is the steel plate, and does it stop in a predictable orientation?
- ☐ Which camera or sensor detects the red LED on the real robot?

Milestones

Milestones are ordered, not dated; dates get set once the team's deadline is known.

1. **M1 Workspace:** ROS 2 workspace, packages, core library skeleton, unit tests running.

2. **M2 Go-to-point in sim:** the D9 reaches typed-in targets in Gazebo using fake drone positions and gyro heading.
3. **M3 Tasks in sim:** clear and tow state machines complete, fake server drives them end to end.
4. **M4 Enemy detection in sim:** red object detected, enemy report under the latency target.
5. **M5 Hardware drive:** core library ported to the ESP32; the real D9 drives to targets from laptop commands.
6. **M6 Hardware tasks:** magnet, camera and push pattern working on the real robot.
7. **M7 Integration:** bridge node on the real server protocol, fakes replaced by real teams.
8. **M8 Berm (FR-7):** scoop, carry and dump, once the bucket mechanism exists.